

Harmonic Numbers and Their Computational Complexity Analysis

Doruk Üstündağ

İzmir Yüksek Teknoloji Enstitüsü

AAW II - İzmir, 10.07.2026



1 Harmonic Numbers

- Generalized Harmonic Numbers
- $J(p)$ Sets

2 Computational Approach on Harmonic Numbers

- Naive Approach
- Binary Splitting
- A New Algorithm For Computation of H_n

3 References

Harmonic Numbers

Definition (Harmonic Series)

The *harmonic series* is defined by the infinite sum

$$\sum_{k=1}^{\infty} \frac{1}{k} = 1 + \frac{1}{2} + \frac{1}{3} + \frac{1}{4} + \cdots .$$

This series diverges.

Adding the first n terms of this series gives us a partial sum:

Definition (Harmonic Numbers - H_n)

Let n be a positive integer. n^{th} *harmonic number* H_n is defined as

$$H_n = 1 + \frac{1}{2} + \frac{1}{3} + \cdots + \frac{1}{n} = \sum_{k=1}^n \frac{1}{k}.$$

Some Important Properties of Harmonic Numbers

One of the most important properties of harmonic numbers H_n is their nonintegrerness when $n > 1$ (It is trivial that $H_1 = \frac{1}{1} = 1$). **Theisinger (1915)** [11] gave an elementary arithmetic proof of this theorem, while an approach relying on Bertrand's postulate was given later. A different point of view given by **Kürschák (1918)** [9] used theory of p -adic valuations to prove nonintegrerness.

Theorem (Nonintegrerness of H_n)

For $n \geq 2$, $H_n \notin \mathbb{Z}$.

Theorem (Babbage, 1819 [2])

For each odd prime p , $H_{p-1} \equiv 0 \pmod{p}$.

Theorem (Wolstenholme, 1862 [13])

For each prime $p \geq 5$, $H_{p-1} \equiv 0 \pmod{p^2}$.

Some Generalization Effort: Generalized Harmonic Numbers

Definition (Generalized Harmonic Numbers)

The n^{th} *generalized harmonic number of order m* is given by

$$H_{n,m} = \sum_{k=1}^n \frac{1}{k^m}.$$

Remark.

The case $m = 1$ reduces to standard harmonic numbers directly. More importantly, we can see generalized harmonic numbers as partial sums of the Riemann zeta function: The limit of $H_{n,m}$ as $n \rightarrow \infty$ is finite if $m > 1$, with the generalized harmonic number bounded by and converging to the *Riemann zeta function*

$$\lim_{n \rightarrow \infty} H_{n,m} = \zeta(m).$$

$J(p)$ Sets and a Conjecture (I)

Eswarathasan & Levine, 1991 [4]

They studied divisibility of harmonic numbers $H_n = a(n)/b(n)$ (in lowest terms) by a prime p , defining

$$J(p) = \{n \geq 0 : a(n) \equiv 0 \pmod{p}\}.$$

They proved $J(p)$ **finite** for $p < 11$, noting even $J(11)$ seemed **difficult**.
Exceptional small cases:

$$J(2) = \{0\}, \quad J(3) = \{0, 2, 7, 22\}.$$

For $p > 3$:

$$\{0, p-1, p(p-1), p^2-1\} \subset J(p).$$

They conjectured that: **For all primes p , $J(p)$ is finite.**

$J(p)$ Sets and a Conjecture (II)

Boyd, 1994 [3]

Boyd studies these sets using p -adic methods and shows that their recursive structure can be modeled by a **critical branching process** (probabilistic and computational way to approach the topic), suggesting that J_p is typically finite but can occasionally be very large. The paper also gives large-scale computations of J_p and finiteness of all $J(p)$ sets with $p < 550$ with three exceptions $p = 83, 127, 397$.

The conjecture remains open. For deeper analysis of p -integral harmonic sums, see **Altuntaş's** "*On the finiteness of some p -divisible sets*" [1], **Göral and Sertbaş's** "*Applications of class numbers and Bernoulli numbers to harmonic type sums*" [7], and **Sanna's** work "*On the p -adic valuation of harmonic numbers*" [10]. For weaker variants of the $J(p)$ sets and conjectures equivalent to the conjecture, consult the concluding remarks in **Göral and Sertbaş's** "*Divisibility Properties of Hyperharmonic Numbers*" [6].

FLINT: Fast Library for Number Theory

FLINT (**F**ast **L**ibrary for **N**umber **T**heory) [5] is a C library, started by **David Harvey** and **William Hart** in 2007, offering highly optimized exact arithmetic for integers, rationals, polynomials, and finite fields. One can use this library in Python as **"python-flint"**.

Key Features

- Since FLINT 3 (2023): unifies former Arb, Antic, and Calcium libraries
- **Exact rational** and p -adic arithmetic, primality testing, factorization
- Distributed with **SageMath** via a Cython wrapper
- Maintained by **Fredrik Johansson** and Albin Ahlbäck

Saturday, February 21, 2009

How (not) to compute harmonic numbers

The n th [harmonic number](#) is the n th partial sum of the divergent harmonic series, $H_n = 1 + 1/2 + 1/3 + \dots + 1/n$. The simplest way to compute this quantity is to add it directly the way it is written: 1, 1+1/2, 1+1/2+1/3, and so on. For n approximately greater than 10 or 100, this is algorithm is not a very good one. Why?

Firstly, in floating-point arithmetic, it is much better to use the asymptotic expansion

$$H_n = \ln n + \gamma + \frac{1}{2}n^{-1} - \frac{1}{12}n^{-2} + \frac{1}{120}n^{-4} + \mathcal{O}(n^{-6})$$

which requires *fewer* terms the larger n is. Combining this with recurrence when n is too small gives a fast numerical algorithm for fixed-precision or arbitrary-precision computation of harmonic numbers (especially large ones). It allows one to determine, for example, that it takes exactly 15092688622113788323693563264538101449859497 terms until the partial sums of the harmonic series exceed 100. (This algorithm is implemented in [mpmath](#), and most other comparable software.)

Secondly, the naive algorithm is especially bad for *exact* computation of harmonic numbers. This might not be obvious at a glance. We are just adding n numbers: what's to improve? The catch is that rational arithmetic has hidden costs: a single addition requires three multiplications and usually a GCD reduction. In contrast with integer or floating-point addition, rational addition gets very slow when the numerators and denominators get large. Computing harmonic numbers the naive way triggers worst-case behavior of rational arithmetic: the denominators grow like $n!$ (if no GCD is performed, the denominator of H_n literally becomes $n!$).

Figure: A Personal Blog Page by **Frederik Johansson** [8]

How To Compute Harmonic Numbers

A baby step for constructing a computational approach is to directly compute the series:

Naive Approach

```
from flint import fmpq
def H(n):
    total = fmpq(0)
    for i in range(1, n + 1):
        total += fmpq(1, i)
    return total
```

With **Schönhage–Strassen multiplication algorithm (FFT-Based)** which has the computational complexity $O(n \log n \log \log n)$, total complexity of the **naive algorithm** is:

$$O(n^2 \log n \log \log n)$$

Divide and Make It Faster (I)

Binary splitting is a divide-and-conquer technique for exact evaluation of rational series.

Setup

For a series

$$S(a, b) = \sum_{n=a}^b \frac{p_n}{q_n},$$

the goal is to find integers $P(a, b)$, $Q(a, b)$ such that $S(a, b) = \frac{P(a, b)}{Q(a, b)}$.

Divide and Make It Faster (II)

Recursive Splitting

Set $m = \left\lfloor \frac{a+b}{2} \right\rfloor$. Then $P(a, b)$ and $Q(a, b)$ are computed recursively from

$$P(a, m), P(m, b), Q(a, m), Q(m, b),$$

combined via a merge step. When $b - a$ is small enough, compute directly from $p_a, \dots, p_b$ and $q_a, \dots, q_b$.

Merge Step

$$\frac{P(a, m)Q(m, b) + P(m, b)Q(a, m)}{Q(a, m)Q(m, b)}$$

Divide and Make It Faster (III)

Why It's Faster

Naive summation requires a full-precision division *per term*. Binary splitting needs only **one** division at the end—faster and free of intermediate rounding error. Thus, cost drops to $O(\log(n) M(n))$ where $M(n)$ is the multiplication cost.

With the help of FLINT we can implement a very fast working binary splitting model to harmonic numbers but since we are again working with a rational series and we have many $Q(a, m_1), \dots, Q(m_{n-1}, b)$ as the denominators of the **leaf nodes**.

Is it really efficient to multiply all of denominator Q s?

Binary Splitting Harmonic Numbers (I)

As Johansson gave an old implementation of the **binary splitting idea** on harmonic numbers computation, we give a much more clean and modern version of the algorithm:

Binary Splitting Algorithm (I)

```
def _harmonic_bs_seq_fmpz(a, b):  
    n = b - a  
    if n == 1:  
        return fmpz(1), fmpz(a)  
    if n <= 16:  
        q = fmpz(1)  
        for i in range(a, b):  
            q *= i  
        p = fmpz(0)  
        for i in range(a, b):  
            p += q // i  
        return p, q  
    m = a + (n >> 1)  
    p1, q1 = _harmonic_bs_seq_fmpz(a, m)  
    p2, q2 = _harmonic_bs_seq_fmpz(m, b)  
    return p1 * q2 + p2 * q1, q1 * q2
```

Binary Splitting Algorithm (II)

```
def harmonic_sequential_fmpz(n):  
    n = int(n)  
    if n == 0:  
        return fmpz(0)  
    p, q = _harmonic_bs_seq_fmpz(1, n + 1)  
    return fmpz(p, q)
```

Again using **Schönhage–Strassen multiplication algorithm (FFT-Based)** and the complexity calculation from **binary splitting** we have the total complexity:

$$O(n \log^2 n \log \log n)$$

Parallelizing Binary Splitting

Algorithm (Ü., 2026 [12])

```
def harmonic_parallel_fmpz(n, workers=None):
    n = int(n)
    if n == 0:
        return fmpz(0)
    workers = workers or os.cpu_count() or 1
    workers = max(1, min(workers, n))
    chunk_size = n // workers
    bounds = [1 + i * chunk_size for i in range(workers)]
    bounds.append(n + 1)
    bounds[-1] = n + 1
    with ProcessPoolExecutor(max_workers=workers) as ex:
        futures = [
            ex.submit(_harmonic_bs_seq_fmpz, bounds[i], bounds[i +
                1])
            for i in range(workers)
        ]
        results = [f.result() for f in futures]
    p, q = results[0]
    for p2, q2 in results[1:]:
        p, q = p * q2 + p2 * q, q * q2
    return fmpz(p, q)
```

Speed Comparison

CPU Spec: AMD Ryzen 9 8940HX: 16C/32T Zen 4 mobile CPU, 2.4–5.3 GHz, and process worker count is set to 4 for parallel computation because of the pooling inefficiencies.

n	Naive (s)	Sequential BS (s)	Parallel BS (s)
10^3	0.0006	0.0004	0.9644
10^4	0.0172	0.0043	1.2662
10^5	0.9881	0.0702	1.2991
10^6	91.1188	1.0596	1.0526
10^7	–	16.8485	10.5951
10^8	–	272.7281	155.7067

Table: Performance comparison of naive, sequential, and parallel computations

How Not To Compute Harmonic Numbers

Question / Remark (Johansson).

Binary splitting is not the fastest algorithm for computing factorials. The fastest known method is to decompose $n!$ into a product of prime powers and perform balanced multiplication-exponentiation. Can this idea be applied to harmonic numbers? [8]

Mainly we have three important questions:

- Can we find a new method as suggested?
- Can we implement it?
- Would it be faster than **binary splitting**?

An Algorithm Proposal

Let us firstly give an important identity for least common multiple of natural numbers up to some natural number n :

$$\text{lcm}\{1, \dots, n\} = \prod_{p \leq n} p^{\lfloor \log_p n \rfloor},$$

where p is a prime number.

Algorithm Proposal (Ü., 2026 [12])

$$H_n = \frac{\sum_{i=1}^n \frac{\prod_{p \leq n} p^{\lfloor \log_p n \rfloor}}{i}}{\prod_{p \leq n} p^{\lfloor \log_p n \rfloor}}$$

Just multiplying and dividing by the least common multiple term we reduce the computation of n^{th} harmonic number H_n to just an integer sum (since $i \mid \text{lcm}\{1, \dots, n\}$, when $i \in \{1, \dots, n\}$) and a mere prime sieving algorithm.

Implementation and optimization of the proposed algorithm is rather trickier than algorithms before since it has a bit more **complex** subroutines than the other ones. But with again a binary splitting type argument at the integer sum we can again get to the computation complexity of binary splitting algorithm for harmonic numbers (nearly $O(n \log^3 n)$).

Implementation and a better optimization on the idea of the new algorithm is still a work in progress! We can layout a roadmap for future works on this algorithm as:

- Finding a better algorithm for the hidden **gcd-trap** in the summation (FLINT's own algorithm is used!)
- Finding a faster method than binary splitting for the integer sum
- Optimizing prime sieving (now using **prime_sieve**)



Ç. Altuntaş, “On the finiteness of some p -divisible sets,” *Communications Faculty of Sciences University of Ankara Series A1 Mathematics and Statistics*, vol. 73, pp. 1011–1039, 2024. DOI: 10.31801/cfsuasma.1441894.



C. Babbage, “Demonstration of a Theorem Relating to Prime Numbers,” *Edinburgh Philosophical Journal*, vol. 1, pp. 46–49, 1819.



D. W. Boyd, “A p -adic Study of the Partial Sums of the Harmonic Series,” *Experimental Mathematics*, vol. 3, pp. 287–302, 1994.
Available at: <https://api.semanticscholar.org/CorpusID:17442396>.



A. Esvarathan and E. Levine, “ p -Integral harmonic sums,” *Discrete Mathematics*, vol. 91, no. 3, pp. 249–257, 1991. DOI: 10.1016/0012-365X(90)90234-9.

-  The FLINT team, “FLINT: Fast Library for Number Theory,” 2026, version 3.6.0. [Online]. Available: <https://flintlib.org>.
-  H. Göral and D. C. Sertbaş, “Divisibility properties of hyperharmonic numbers,” *Acta Mathematica Hungarica*, vol. 154, no. 1, pp. 147–186, 2018. DOI: 10.1007/s10474-017-0766-7.
-  H. Göral and D. Sertbaş, “Applications of class numbers and Bernoulli numbers to harmonic type sums,” *Bulletin of the Korean Mathematical Society*, 2021. DOI: 10.4134/BKMS.b201045.
-  F. Johansson, “How (not) to compute harmonic numbers,” *Fredrik Johansson Blog*, Feb. 20, 2009. [Online]. Available: <https://fredrik-j.blogspot.com/2009/02/how-not-to-compute-harmonic-numbers.html>

-  J. Kürschák, "On the harmonic series," *Matematikai és Fizikai Lapok*, vol. 27, pp. 299–300, 1918.
-  C. Sanna, "On the p -adic valuation of harmonic numbers," *Journal of Number Theory*, vol. 166, pp. 41–46, 2016. DOI: 10.1016/j.jnt.2016.02.020.
-  L. Theisinger, "Bemerkung über die harmonische Reihe," *Monatshefte für Mathematik und Physik*, vol. 26, pp. 132–134, 1915.
-  D. Üstündağ, "A Survey On Harmonic Numbers and Their Algorithmic Analysis," *Preprint*, 2026.
-  J. Wolstenholme, "On Certain Properties of Prime Numbers," *The Quarterly Journal of Pure and Applied Mathematics*, vol. 5, pp. 35–39, 1862.

$$\frac{1}{T} + \frac{1}{H} + \frac{1}{A} + \frac{1}{N} + \frac{1}{K} + \frac{1}{S}$$

THANK YOU ALL FOR LISTENING

E-mail: dorukustundag5@hotmail.com
ResearchGate: Doruk Üstündağ
LinkedIn: Doruk Üstündağ